Lab 7: Iterative Optimization

Due Wednesday, March 25, 11:59pm

In previous labs, you reconstructed images using closed-form solutions such as Wiener deconvolution. While these approaches are fast and convenient, they have important limitations: they cannot easily enforce constraints like non-negativity or incorporate more advanced priors about the image.

In this lab, we move to iterative optimization methods for solving imaging inverse problems. Instead of computing a solution in a single step, we will start from an initial guess and gradually improve it over many iterations. This approach provides much greater flexibility, but it also requires a deeper understanding of how the algorithm behaves.

In order to use convex optimization to solve imaging inverse problems, we need to calculate gradients. Often, we want to minimize the data fidelity term:

$$f(\mathbf{x}) = \frac{1}{2}\|\mathbf{y} - \mathbf{A}\mathbf{x}\|_2^2$$

where $\mathbf{x}$ is the object, $\mathbf{y}$ is the measurement, and $\mathbf{A}$ is the imaging forward model. The gradient of this is:

$$\nabla f(\mathbf{x}) = \mathbf{A}^*(\mathbf{A}\mathbf{x} - \mathbf{y})$$

Using this gradient, we can initialize a guess of $\mathbf{x}^0$ (e.g. all zeros) and iteratively update:

$$\mathbf{x}^{k+1} \leftarrow \mathbf{x}^k - \frac{1}{L}\nabla f(\mathbf{x}^k)$$

where $\frac{1}{L}$ is the step size. $\mathbf{x}^k$ will slowly start off as zero when $k = 0$ and converge to the final estimate $\hat{\mathbf{x}}$ when k is large.

In Lab 6, you reconstructed DiffuserCam images using Wiener deconvolution—a one-step, closed-form solution. Wiener deconvolution is fast, but it cannot enforce constraints like non-negativity or incorporate image priors. In this lab you will implement iterative reconstruction algorithms that are more flexible: gradient descent. This method could be applied to many types of imaging problems, such as tomography, MRI, deblurring, and others.

Allowed libraries. You may use `numpy`, `scipy`, `matplotlib`, `imageio`, and `torch`.

Note on FFT normalization. In this lab we use `norm='ortho'` for all FFT operations. With orthonormal normalization the FFT and IFFT are each other's adjoint (up to conjugation), which simplifies the derivation of $\mathbf{A}^*$. We also apply `ifftshift` to the padded PSF before taking its FFT, so that the zero-frequency component is in the correct position.

Starter code. We provide `inverse_solver.py`, which contains an `InverseSolver` class that structures the reconstruction code you will write. Open this file and read through it before starting. The class constructor, main loop, and helper methods (`pad`, `crop`, `prox`) are already implemented. You will fill in the methods marked `TODO`. You will also extend this class with new priors in Lab 8.

The `InverseSolver` accepts a proximal operator `prox_fn(x,lam,step_size)` for code you'll write in Lab 8. For plain gradient descent, pass `prox_fn=None`.

Until you implement `power_iteration`, pass `step_size` directly to avoid an error.

1 Adjoints and the Adjoint Test

When $\mathbf{A}$ is expressed as a matrix, all we need is to compute $\mathbf{A}^T$, the transpose of $\mathbf{A}$ (or the conjugate transpose if $\mathbf{A}$ is complex), and we can easily compute the gradient. In imaging, due to size limitations, we do not often compute the $\mathbf{A}$ matrix and instead use linear operators to describe this function. In this

case, rather than computing the transpose of the matrix, we instead need to compute the *adjoint*, $\mathbf{A}^*$, of the operator. The adjoint is equivalent to the conjugate transpose of $\mathbf{A}$.

You can think of $\mathbf{A}$ as taking you from object space to sensor space, and $\mathbf{A}^*$ as taking you from sensor space back to image space. An adjoint is defined as:

$$\langle \mathbf{Ax}, \mathbf{y} \rangle = \langle \mathbf{x}, \mathbf{A}^*\mathbf{y} \rangle$$

where $\langle \cdot, \cdot \rangle$ is the inner product of the vector space.

For DiffuserCam, $\mathbf{A}$ is defined as:

$$\mathbf{A}(\mathbf{x}) = \text{crop}(\mathbf{h} * \mathbf{x})$$

Thus, to find the adjoint of $\mathbf{A}$, you will need to find the adjoint of a crop operator and the adjoint of a convolution operator. Once you do this, you can write out the equation for $\mathbf{A}^*$.

Problem 1.1:

(a) Write the equation for the adjoint, $\mathbf{A}^*$, for DiffuserCam's forward model. Then, fill in the `A` and `A_adjoint` methods in `inverse_solver.py`. For now you can ignore the `self.mask` logic (leave it for Lab 8).

Hint: `A` should compute `FFT(x)`, multiply by `self.H`, then `IFFT`, take the real part, and `self.crop()`. `A_adjoint` should `self.pad(y)`, compute `FFT`, multiply by `self.H_conj`, then `IFFT` and take the real part.

(b) Perform the adjoint test to verify your implementation. You can use the provided `adjoint_test(solver,y_shape)` function from `inverse_solver.py`. Print both inner products. If they do not match, your adjoint is incorrect and the rest of the lab will not work.

Hint: your random $\mathbf{x}$ should be the *padded* size (2H×2W×C) and your random $\mathbf{y}$ should be the *measurement* size (H×W×C).

$$A^* = F^{-1}(F(pad(y)) * diag(H))$$

2 Solving Inverse Problems with Convex Optimization

Now that you have the adjoint for your DiffuserCam system, you can solve the inverse problem using optimization. Let's begin by solving an optimization problem which minimizes the data-fidelity term:

$$\min_{\mathbf{x}} \frac{1}{2}\|\mathbf{y} - \mathbf{A}\mathbf{x}\|_2^2$$

Minimizing this function will give us an estimate of $\mathbf{x}$ which is consistent both with the measurement $\mathbf{y}$ and with the forward model $\mathbf{A}$.

2.1 Gradient descent

We can minimize this function by performing the following algorithm:

Input: $\mathbf{x}_0 \in \mathbb{R}^n$ and $L \geq \lambda_{\max}(\mathbf{A}^*\mathbf{A})$

while $\mathbf{x}^k$ not converged $(k = 1, 2, \ldots)$ **do**

$$\mathbf{x}^k \leftarrow \mathbf{x}^k - \frac{1}{L}\mathbf{A}^*(\mathbf{A}\mathbf{x}^k - \mathbf{y})$$

end while

Output: $\hat{\mathbf{x}} \leftarrow \mathbf{x}^k$

Note that L can be calculated by finding the max eigenvalue of $(\mathbf{A}^*\mathbf{A})$. This value can be estimated using the power iteration method. If you don't want to find this value, you can set L to be large. If your loss diverges, you will know that your step-size is too big, so L should be smaller. For faster convergence, you should set your step-size to be just small enough that your loss does not diverge.

We recommend plotting the estimated image and the loss over iterations as you run your gradient descent algorithm. Make sure to plot your loss $(f(\mathbf{x}) = \frac{1}{2}\|\mathbf{y} - \mathbf{A}\mathbf{x}\|_2^2)$ as a function of iterations to check whether your solution is converging. If your loss increases, something is wrong—either your step size is too big, or you have an error in your code.

Problem 2.1.1:

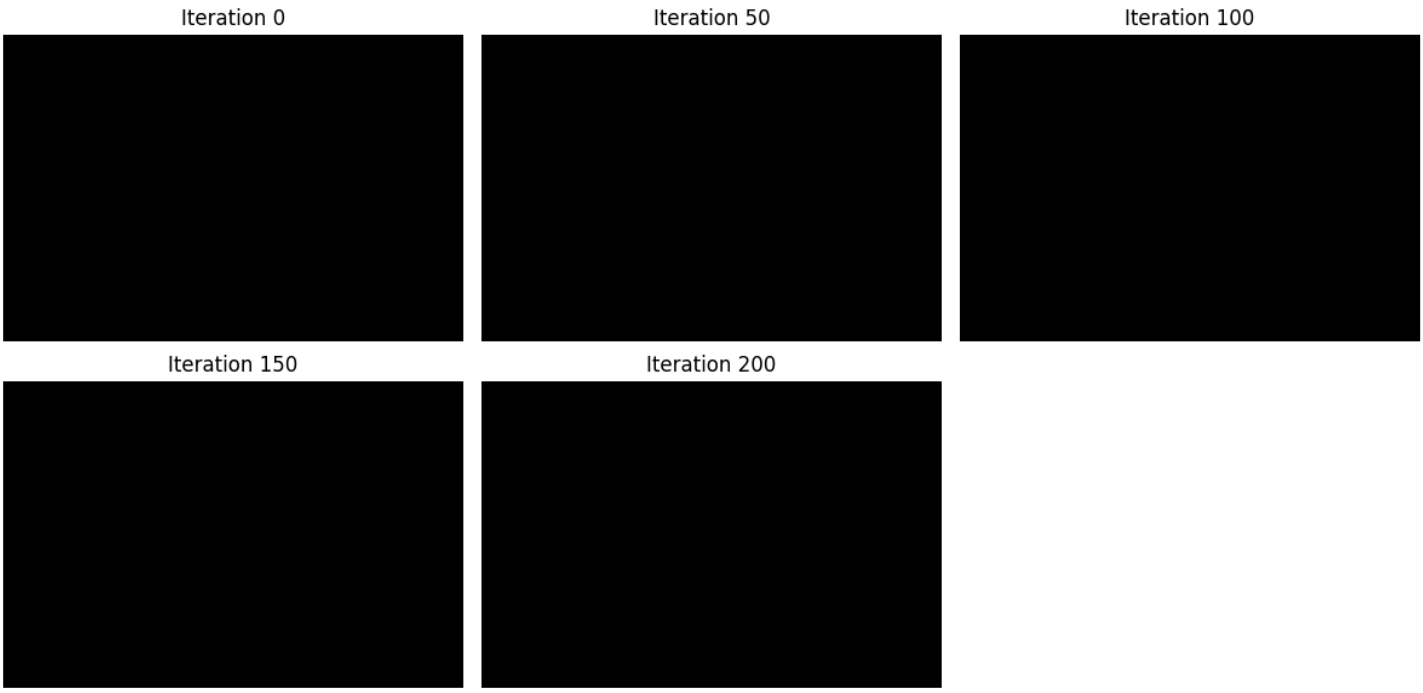
(a) Fill in `gd_update`, and `loss` in `inverse_solver.py`. Read the docstrings for implementation details.

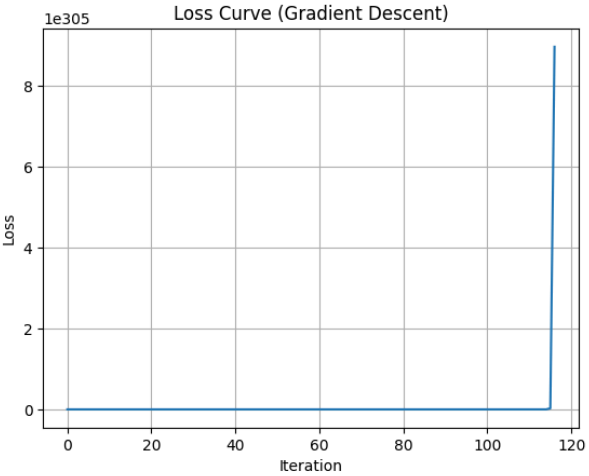
(b) Create an `InverseSolver` with `algorithm='gradient_descent'` and run it on your DiffuserCam measurement for 200 iterations, with step_size = 1000. Display the reconstruction at iterations 0, 50, 100, 150, and 200. Plot the loss curve.

(c) What happened to the loss curve? What do you think is the problem?

(d) Fill in `power_iteration`, in `inverse_solver.py`. Read the docstrings for implementation details. For `power_iteration`, work in the full padded space using `self.H` directly (do *not* include crop/pad). Run it on your DiffuserCam measurement for 200 iterations, without defining the stepsize. Display the reconstruction at iterations 0, 50, 100, 150, and 200. Plot the loss curve.

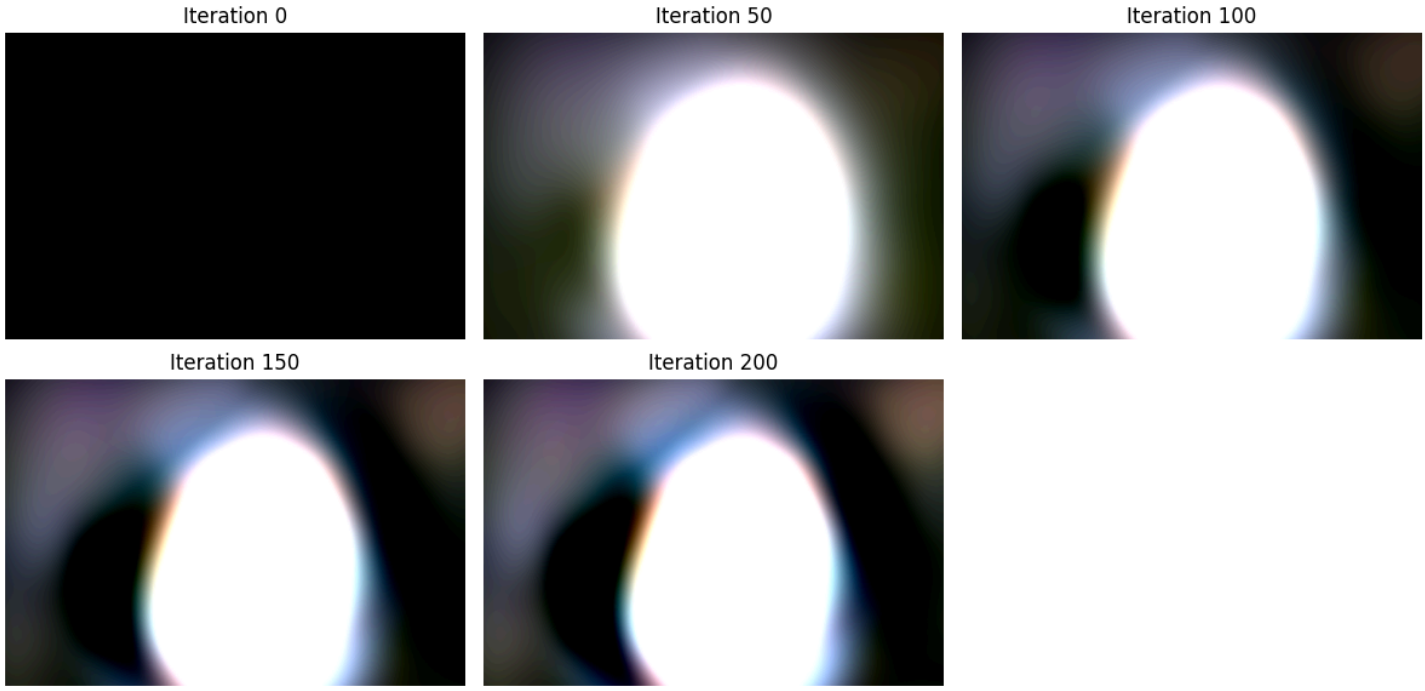
b)

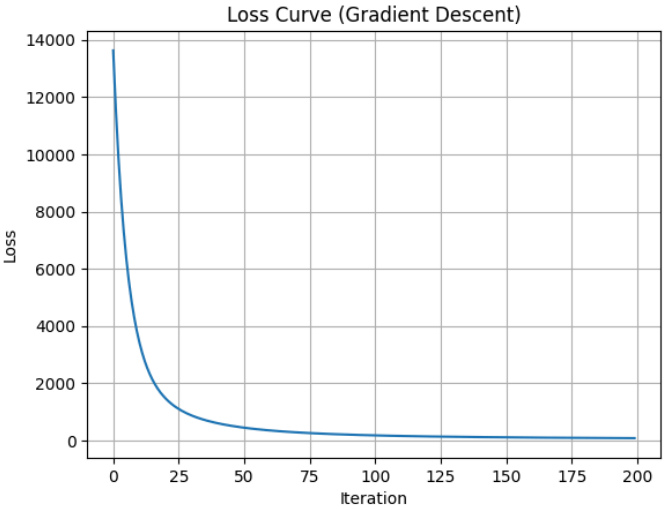




c) The step size was way too big, so the gradient descent was never able to stabilize at the point that minimized loss

d)


Iteration 0
Iteration 50
Iteration 100
Iteration 150
Iteration 200


Loss Curve (Gradient Descent)
Loss
Iteration
14000
12000
10000
8000
6000
4000
2000
0
0
25
50
75
100
125
150
175
200

Checkoff

When you're ready, ask a TA to check you off.

3 PyTorch Backprop

Thus far, we've been manually keeping track of our gradients and using efficient algorithms with good convergence rates to solve our imaging inverse problems. However, to do this, we've had to carefully write-out our adjoints and use priors that nicely fit within our problem structure.

Sometimes, it's hard to determine the adjoint, we may want a quick and dirty solution, or we want to interface with more modern priors. In that case, it's useful to solve our imaging inverse problem using a machine learning library like PyTorch, TensorFlow, or JAX which automatically keeps track of gradients. Using one of these libraries simplifies the problem, since we no longer have to keep track of adjoints. Instead, we can write-out our loss function and use backpropagation to help us update our image estimate to solve our imaging inverse problem.

Note that we no longer need to define an adjoint—all we need is to define our forward model in PyTorch. Then, you can use backprop to solve for the image, `x_hat`.

Setting up PyTorch. PyTorch uses *tensors*, which are similar to numpy arrays but can automatically track computations for gradient calculation. To use PyTorch for our inverse problem, we need to: (1) convert our numpy arrays into PyTorch tensors, (2) create our image estimate `x` as a tensor with `requires_grad=True` so PyTorch records all operations on it, and (3) implement our forward model using PyTorch operations instead of numpy.

Here is how to set things up:

```
import torch

# Convert H to a PyTorch tensor.
# H is complex-valued, so we use complex128.
H_torch = torch.tensor(H, dtype=torch.complex128)

# Convert measurement y to a PyTorch tensor.
y_torch = torch.tensor(y, dtype=torch.float64)

# Create image estimate x as zeros.
# requires_grad=True tells PyTorch to track all
# operations on x so it can compute gradients.
full_shape = (y.shape[0]*2, y.shape[1]*2,
              y.shape[2])
x = torch.zeros(full_shape, dtype=torch.float64,
                requires_grad=True)
```

Next, implement the DiffuserCam forward model using PyTorch's FFT functions. These work the same as numpy's, but use `dim=` instead of `axes=` and access the real part with `.real`:

```
def forward_torch(x, H_torch):
    """DiffuserCam forward model in PyTorch."""
    X = torch.fft.fft2(x, dim=(0,1), norm='ortho')
    out = torch.fft.ifft2(X * H_torch, dim=(0,1),
                          norm='ortho').real
    # crop
    h4 = out.shape[0] // 4
    w4 = out.shape[1] // 4
    return out[h4:-h4, w4:-w4]
```

Computing gradients with backpropagation. To compute gradients, compute the loss and then call `loss.backward()`. This runs backpropagation through all the recorded operations and stores the gradient $\nabla_{\mathbf{x}} f$ in `x.grad`:

```
y_hat = forward_torch(x, H_torch)
loss = 0.5 * torch.sum((y_hat - y_torch)**2)
loss.backward() # computes gradients
print(x.grad.shape) # same shape as x
```

Note that we no longer need the adjoint $\mathbf{A}^*$—PyTorch figures out the gradient automatically by applying the chain rule in reverse through all the FFT and cropping operations.

Gradient descent with autograd. To run gradient descent, repeatedly compute the loss, call `backward()`, and update `x` using the gradient. There are two important details:

- The update step must be inside a `torch.no_grad()` block so that PyTorch does not try to record the gradient update itself (we only want to track operations that contribute to the loss).
- You must call `x.grad.zero_()` after each update because PyTorch *accumulates* gradients by default—each call to `backward()` *adds* to the existing `x.grad` rather than replacing it.

```
losses = []
for i in range(num_iters):
    # Forward pass
    y_hat = forward_torch(x, H_torch)
    loss = 0.5 * torch.sum((y_hat - y_torch)**2)

    # Backward pass: compute gradients
    loss.backward()

    # Update x (don't track this operation)
    with torch.no_grad():
        x.data -= step_size * x.grad
        x.grad.zero_() # clear for next iteration

    losses.append(loss.item()) # .item() -> float

# Convert result back to numpy for plotting
x_hat = x.detach().numpy()
```

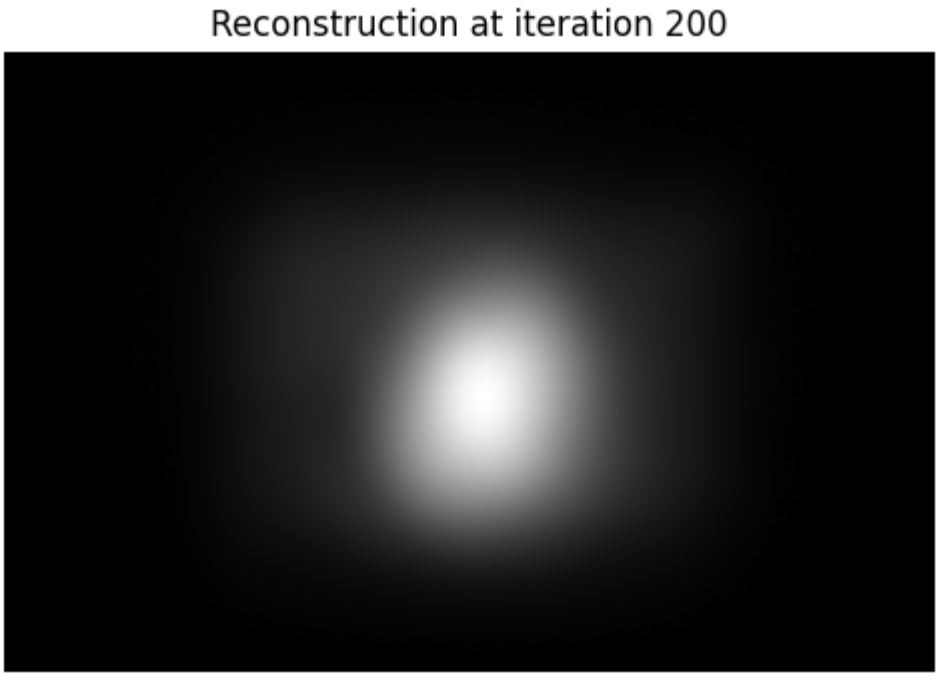
Problem 3.1:

(a) Implement the DiffuserCam forward model in PyTorch using the starter code above. Using a random `x`, compute the gradient via `loss.backward()` and compare `x.grad` (converted to numpy with `x.grad.numpy()`) to the gradient computed using your numpy adjoint, i.e., $\mathbf{A}^*(\mathbf{A}\mathbf{x} - \mathbf{y})$. Report the maximum absolute difference between the two.

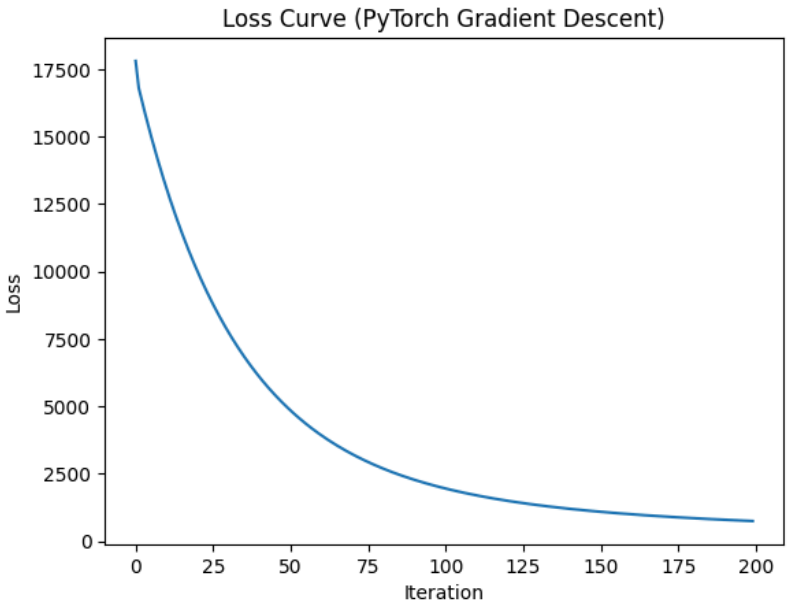
(b) Implement gradient descent using PyTorch autograd (no manual adjoint needed). Run for 200 iterations on your DiffuserCam measurement. Plot the final reconstruction and loss curve.

(c) How does this compare to your result from 2.1? Comment on the speed of convergence.

a) Difference = 4.1633363423443 37e-17



b)



c) This approach has a similar loss curve compared to 2.1, but finished in about half the time.

Submission

Submit the following on Gradescope:

- A single PDF with answers to all problems (Sections 1–3), including all requested figures, code, and discussion. Submit this to "Lab 7 Writeup."
- Your code files (`inverse_solver.py` and any scripts). Include a `README.md` explaining how to run your code. Submit this to "Lab 7 Code."